

Журнал изменений

Диспетчер обращений по авариям и заявкам · чат-бот в MAX · трек «Умный город»

Хакатон MAX · онлайн-этап · рабочий документ команды · репозиторий `EEGRINO/magistral-dispatcher-max`
(приватный, переименован 21.09.2026 из `-MAX-`)

Как читать. Дни нумеруются относительно дедлайна: Д-0 — день сдачи, Д-10 — за десять дней до него. Внутри дня изменения сгруппированы по типу: **добавлено** **изменено** **исправлено** **удалено / отменено** . Отдельным разделом вынесены **отменённые решения** — то, что было сделано, а потом сознательно переиграно, с причиной. Время коммитов — по часам машины сборки.

Д-10 · Допуск: бот отвечает в MAX

ИТОГ ДНЯ С нуля до работающего бота в MAX плюс инфраструктура запуска одной командой.

Почему именно это первым. По методичке проверка бота в MAX — допуск, а не функция: если бот не отвечает, решение получает 0 баллов и дальше не оценивается. Поэтому первым коммитом шёл рабочий `/start` , а не архитектурный скелет.

добавлено

Что	Зачем именно так
Бот: <code>/start</code> и эхо через MAX Bot API, long polling	Long polling не требует публичного HTTPS-домена и туннеля — бот поднимается где угодно, на демо меньше точек отказа
Свой тонкий клиент MAX на <code>fetch</code> , без SDK	В июле 2026 MAX сменил домен на <code>platform-api2.max.ru</code> и запретил токен в query. Проверенный npm-SDK всё ещё ходил на мёртвый адрес
Обработка двух событий старта: <code>bot_started</code> и текстового <code>/start</code>	Это разные события MAX. При нажатии кнопки «Старт» <code>message_created</code> не приходит вовсе — без отдельной ветки бот молчал бы на первом же экране
<code>docker-compose.yml</code> : bot, api, db	Запуск одной командой — требование сдачи. У БД healthcheck через <code>pg_isready</code> , а не проверка порта: порт открывается раньше готовности базы
Защита от секретов: <code>.githubhooks/pre-commit</code> + <code>scripts/check-secrets.sh</code>	Блокирует коммит с токеном, ключом или файлом <code>.env</code> . Хук лежит в репозитории, а не в <code>.git/hooks</code> , — достаётся всей команде
<code>bot/certs/</code> — цепочка Russian Trusted CA	См. «исправлено» ниже. Публичный сертификат, не секрет
<code>.gitattributes</code>	Без него Windows-git отдаёт POSIX-скрипты с CRLF, и <code>pre-commit</code> ломается при клонировании на Linux/Mac

Документация: <code>CONTRIBUTING.md</code> , <code>docs/max-notes.md</code> , <code>docs/RUN.md</code> , <code>docs/EXPLANATORY-NOTE.md</code>	<code>max-notes.md</code> — единственный источник правды по MAX Bot API: материалы в интернете описывают API до миграции и возвращают 401
--	---

изменено

удалено

- Из рабочей копии `.env.example` **убран реальный токен**, файл заменён на плейсхолдеры; рабочее значение перенесено в локальный `.env` (он в `.gitignore`). В git токен при этом не попадал — см. раздел «Аудит секретов».
- `.gitignore` дополнен разделами: секреты, артефакты сборки, файлы IDE, дампы БД.

исправлено

Симптом	Настоящая причина и решение
Бот не стартовал: <code>fetch failed</code> на каждой попытке опроса	<code>platform-api2.max.ru</code> подписан цепочкой Russian Trusted CA (Минцифры РФ). Windows и <code>curl</code> ей доверяют, встроенный список CA у Node.js — нет, отсюда <code>UNABLE_TO_GET_ISSUER_CERT_LOCALLY</code> . Сертификат снят с боевого сервера, проверен <code>openssl verify</code> и подключается через <code>NODE_EXTRA_CA_CERTS</code>
Сертификат молча не попадал в коммит	Правило <code>*.pem</code> в <code>.gitignore</code> , добавленное против приватных ключей, отсекало и легитимный публичный сертификат. Добавлено точечное исключение для <code>bot/certs/*.pem</code>
Переменная задана, но TLS всё равно падает	<code>node --env-file</code> не подходит для <code>NODE_EXTRA_CA_CERTS</code> : она читается нативным TLS-слоем раньше, чем <code>--env-file</code> успевает записать её в <code>process.env</code> . Передаётся через <code>cross-env</code> . В Docker ловушки нет — <code>compose</code> кладёт переменные в окружение до старта процесса

Коммит	Описание	Время
95c28e9	feat(bot): рабочий /start и эхо через MAX Bot API	15:54
d9df482	fix(bot): доверять Russian Trusted CA, иначе fetch к MAX падает	16:31
3112b7b	docs: гайд по запуску и пояснительная записка	16:40

Проверено: бот ответил на `/start` и эхо реальным пользователям в MAX; `docker compose up` поднимает все три сервиса одной командой.

Д-9 · КТ-1: сквозной провод МАХ → бот → API → БД

ИТОГ ДНЯ Контрольная точка закрыта: цепочка работает целиком и подтверждена живым сообщением в МАХ.

добавлено

Что	Зачем именно так
Схема БД: <code>organizations</code> , <code>houses</code> , <code>routing_rules</code> , <code>residents</code> , <code>tickets</code> , <code>ticket_events</code>	Состав таблиц и полей задан разработчиком и согласован. Плюс enum-типы, индексы и триггер на <code>updated_at</code>
Миграции на <code>node-pg-migrate</code> , а не <code>knex</code>	<code>Knex</code> — query builder, он появился бы в проекте только ради миграций и создал бы вторую точку правды о схеме. Тело миграции — сырой SQL, читается без знания JS
Одноразовый сервис <code>migrate</code> в <code>compose</code>	Схема применяется до старта <code>api</code> (<code>service_completed_successfully</code>). Отдельным сервисом: видно в выводе, не гонка при нескольких репликах, перезапуск без трогания <code>api</code>
<code>db/seed.js</code> — тестовые организации и дома	Идемпотентный, помечен как тестовый, автоматически не запускается: подсовывать выдуманные данные в любую поднятую БД нельзя
API: <code>POST /residents/find-or-create</code> , <code>POST /tickets</code> , <code>GET /tickets/:id</code>	Минимум под сквозной проход. <code>find-or-create</code> идемпотентен через <code>ON CONFLICT</code> — два быстрых <code>/start</code> подряд не создадут дубль
Валидация на <code>TypeBox</code> , а не <code>Zod</code>	<code>TypeBox</code> даёт обычный JSON Schema: Fastify одной схемой и проверяет вход, и сериализует ответ, и выводит типы. <code>Zod</code> потребовал бы адаптер
Единый конверт ошибок с машиночитаемым <code>code</code>	Клиент ветвится по <code>code</code> , а не по тексту сообщения
<code>bot/src/api-client.ts</code>	Отдельно от клиента МАХ: это разные внешние миры с разной политикой ошибок. Свой таймаут, чтобы зависший <code>api</code> не подвесил цикл long polling
<code>docs/api.md</code> — контракт API	На него завязан клиент мини-приложения. Меняется эндпоинт — правится тем же коммитом

изменено

- `GET /health` теперь проверяет и соединение с БД, при недоступной отвечает 503. На него завязан healthcheck `compose`, а от него — порядок запуска бота.

- **Приветствие бота содержит** `resident.id` — это и есть доказательство сквозного прохода. Временная логика: полноценный сценарий диалога придёт на Д-7.
- `api/src/index.ts` : заглушка заменена на bootstrap с корректной остановкой (сначала перестаём принимать запросы, потом закрываем пул соединений).
- Порядок запуска в compose: `db (healthy) → migrate (код 0) → api (healthy) → bot`.
- `db/README.md` переписан: команды миграций и правило «применённые миграции не редактируем».

исправлено

Симптом	Настоящая причина и решение
Битый JSON в запросе отдавал <code>500 internal_error</code>	Обработчик схлопывал в 500 ошибки самого Fastify, у которых есть собственный <code>statusCode: 400</code> . По такому ответу клиент искал бы проблему на сервере вместо своего запроса. Теперь 4xx отдаётся как есть, с кодом <code>bad_request</code>

Коммит	Описание	Время
1202912	feat: KT-1 — сквозной провод MAX → бот → api → БД	20:27

Проверено: на снесённом томе поднято с нуля — порядок запуска, применение миграций, все эндпоинты включая ошибочные пути, идемпотентность `find-or-create` и `seed`. Живой `/start` в MAX вернул Ваш ID: 1; запись в `residents` подтверждена запросом `psql`.

Отменённые решения

Раздел для тех, кто вернётся к коду через неделю: здесь то, что было сделано осознанно, а потом так же осознанно переиграно. Чтобы никто не «чинил» обратно.

Решение	Когда и почему отменено
У сервиса <code>bot</code> намеренно нет <code>depends_on</code> принято на Д-10	Отменено на Д-9. На Д-10 бот ни от чего не зависел, и привязка к чужому <code>healthcheck</code> означала бы «бот молчит из-за упавшей БД» — худшее, что может случиться на защите. На Д-9 зависимость стала настоящей: на <code>/start</code> бот идёт в <code>api</code> за жителем. Исходный риск при этом никуда не делся и снят в коде, а не в compose: если <code>api</code> не ответил, бот здороваётся без ID, но не молчит. <code>depends_on</code> управляет только порядком старта и не защищает от падения <code>api</code> через час после запуска.

Заглушка <code>GET /health</code> без проверки БД принято на Д-10	Заменено на Д-9. Пока БД не использовалась, хватало ответа «процесс жив». После появления схемы такой healthcheck стал врать: api мог считаться healthy, не видя базу, и бот стартовал бы в пустоту.
---	---

Аудит секретов

Результат: токенов и ключей в истории репозитория нет. Поиск выполнен по всем объектам всех коммитов и веток — 0 совпадений.

На Д-10 было высказано предположение, что токен бота попал в первый коммит и был запущен. **Предположение оказалось неверным.** В коммите `1a55d02` строка выглядела как `BOT_TOKEN=` — с пустым значением. Реальный токен существовал только в рабочей копии файла на диске и был заменён плейсхолдерами раньше, чем попал в индекс.

Практический вывод: **перевыпускать токен не требуется**, переписывать историю git не требуется. Задача «аудит истории на секреты», стоявшая в плане на Д-3, выполнена досрочно и закрыта. Повторить проверку в любой момент можно командой из `scripts/check-secrets.sh --tracked`.

Что дальше

День	Этап	Результат, по которому будем считать готовым
Д-8	Данные и правила маршрутизации	<code>resolve(type, place, houseId)</code> возвращает ответственного, срок и ссылку на норматив; добавление строки в конфиг меняет поведение без пересборки образа
Д-7	Основной сценарий бота	Сценарий жителя проходит от QR-кода до номера заявки в мобильной и веб-версии MAX
Д-5	Мини-приложение диспетчера	Диспетчер меняет статус, изменение видно в списке после перезагрузки
Д-4	Замыкание петли уведомлений	Смена статуса → житель получает уведомление в боте за секунды

Открытый вопрос к Д-7. Сейчас житель заводится в БД только по `/start` . Человек, начавший диалог обычным сообщением, в базу не попадает — в логах такой чат уже наблюдался. Заводить жителя на любом первом сообщении или оставить только по `/start` — это поведение диалогового автомата, решается вместе со сценарием.

Журнал обновляется в конце каждого дня разработки. Источник — `docs/changelog.html` , PDF пересобирается командой из `docs/RUN.md` . Актуальный список изменений также продублирован на главной странице репозитория (`README.md`).